

PRE-CLASS READING 4 OF 5

Your AI Product Needs Evals

Husain, H. (2024). "Your AI Product Needs Evals." *hamel.dev*. Published 29 March 2024.

ASSIGNED READING

Full post — through the three levels of evaluation ~20 min

The post has short code / tool-UI examples — read them for the idea, not the syntax included

KEY TAKEAWAY

What separates a real LLM product from a demo is not a better prompt — it is an evaluation system. Success is a function of how fast you can iterate, and you can only iterate fast if you can (1) measure quality, (2) debug by looking at data, and (3) change behaviour. Most people only do #3. Build the eval loop and everything else becomes easy.

WHY IT MATTERS FOR THIS COURSE

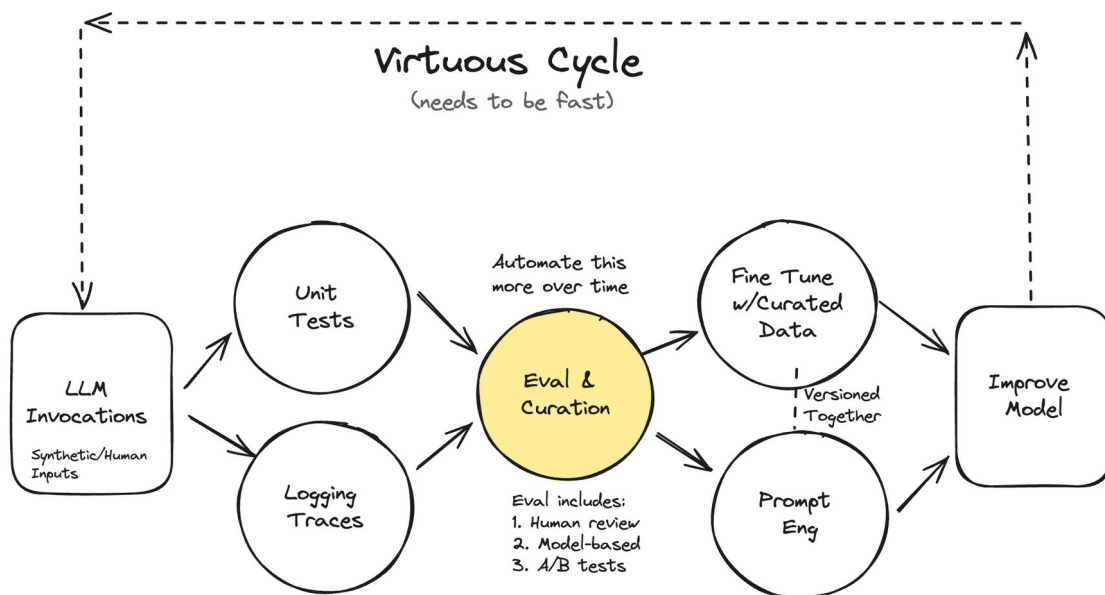
Class 2's C4 capsule treats evaluation as a first-class stack layer, and this post is its backbone. In the RAG Colab you'll run the same query through the local pipeline and the API one — the way you'll actually tell them apart (quality, not vibes) is exactly the 'look at your data' discipline described here. It is also the operational root of A1's prompt + 10-case eval harness, released at the end of this week, and it pays off the warning in Reading 3 (Software 2.0 fails silently, so you must measure it).

Feeds Class 2: C4 (evals as a layer) · the RAG Colab (local vs API comparison) · A1 (10-case eval harness).

MAIN POINTS

Iterating quickly == success

- **The three activities.** Evaluate quality, debug by inspecting data, and change behaviour (prompting, fine-tuning, code). People fixate on the third; doing all three creates the virtuous cycle that separates great AI products from mediocre ones.



The 'virtuous cycle' (from the post): logging traces + unit tests feed Eval & Curation, which drives fine-tuning and prompt improvements — and it has to be fast. Source: *hamel.dev*.

The case study: Lucy (a real-estate assistant)

- **The plateau.** Prompt engineering got Lucy going, then stalled: fixing one failure spawned another (whack-a-mole), no visibility beyond 'vibe checks,' and prompts bloated with edge cases. The fix was a systematic eval system at the centre.

Three levels of evaluation (cost L3 > L2 > L1)

- **Level 1 — Unit tests.** Assertions, scoped by feature and scenario (e.g. 'listing finder returns exactly one result'), plus generic checks (e.g. 'never leak an internal UUID'). Cheap, run on every change. Synthesise test inputs with an LLM. Pass rate is a product decision, not necessarily 100%.

- **Level 2 — Human & model eval.** Log traces (a full record of a conversation), then remove ALL friction from looking at them — often by building a domain-specific viewer. Start with binary good/bad labels. Then use an LLM-as-judge, aligned to a human and tracked by agreement (or precision/recall when classes are imbalanced).
- **Level 3 — A/B testing.** Run in production once the product is mature enough to show real users. Reserved for significant changes.

Eval systems unlock ‘superpowers’ for free

- **Fine-tuning.** 99% of fine-tuning is assembling high-quality data; a good eval system is already a data-curation engine (fine-tuning teaches syntax / style / rules, while RAG supplies facts and context).
- **Debugging.** The same traces, assertions, and search tools you built for evaluation are exactly what you need to find and fix a production error fast.

The takeaways he wants you to keep

- Remove all friction from looking at data; you’re doing it wrong if you aren’t looking at lots of it. Keep it simple — don’t buy fancy tools first. Don’t rely on generic eval frameworks; build tests specific to your problem. Write many tests and update them constantly. Use LLMs to bootstrap the whole eval system.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall.

1. Husain’s number-one rule is ‘you are doing it wrong if you aren’t looking at lots of data.’ When you last used an AI tool, did you actually read its outputs closely — or trust the vibe? What might you have caught if you’d read 50 of them?
2. Level 1 evals are just assertions — ‘did it return exactly one listing?’, ‘did it leak an ID it shouldn’t?’. Pick a task in your field and write two assertions you could check automatically.
3. He warns against generic, off-the-shelf ‘is this answer good?’ scores in favour of problem-specific tests. Why might a generic score mislead you about your own product’s quality?

SOURCE LINKS

Post · <https://hamel.dev/blog/posts/evals/>

Origin · Vanishing Gradients podcast conversation (linked in the post)

Related · Jason Liu, ‘Levels of Complexity: RAG Applications’ (linked for evaluating RAG)

CAVEATS

- **1.** This summary is not a substitute for the reading. It is a map, not the territory. Read the source — it is the authority.
- **2.** This summary was generated and fact-checked using Anthropic models against the original hamel.dev post. The diagram is reproduced from the post for classroom use. Flag anything that looks wrong.